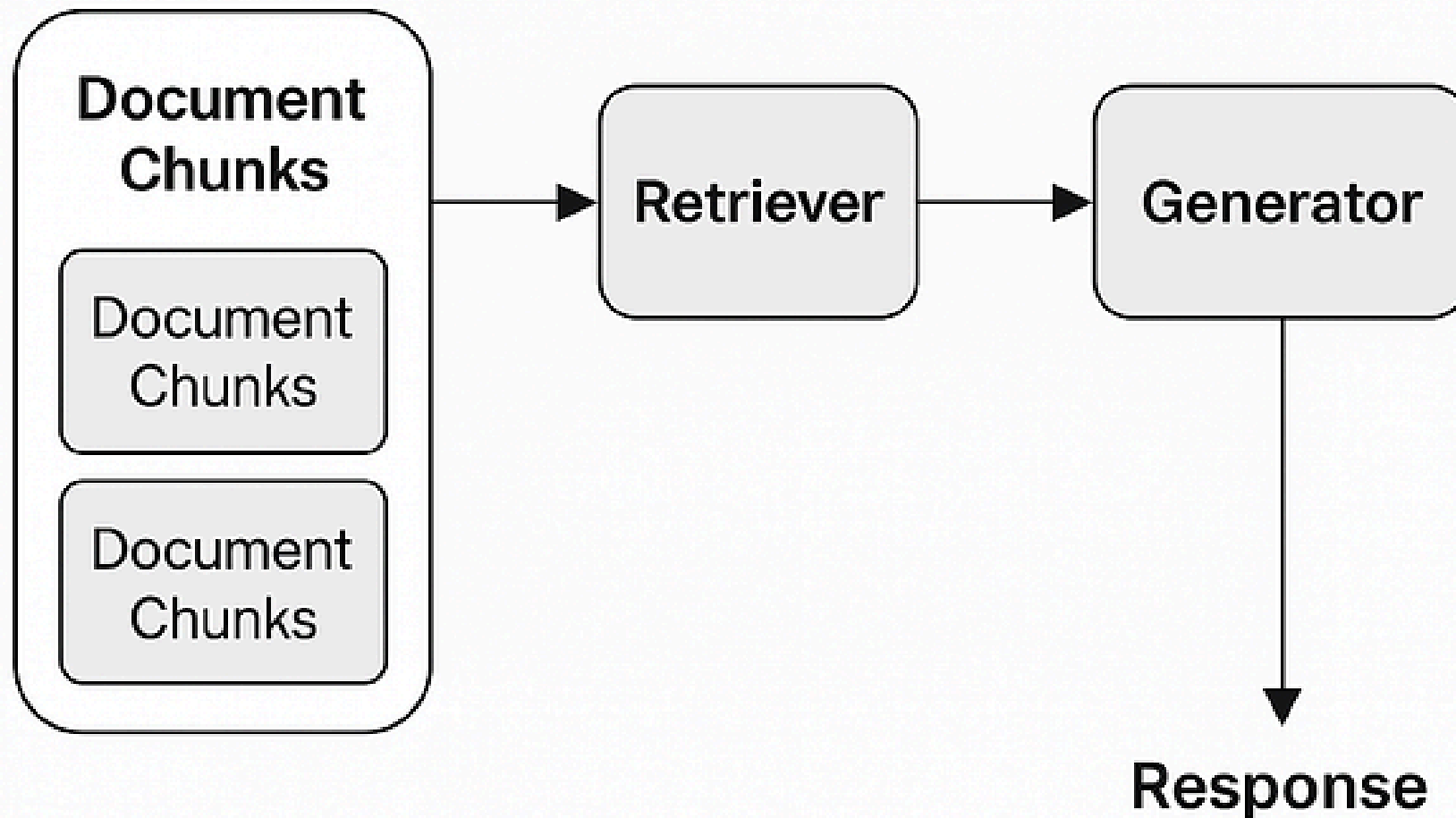




Standard RAG



25 Types of RAG—Which One Fits Your Project Best?

“What if your AI could pull the perfect answer from the world’s knowledge every single time?”



Khushbu Shah

15 min read

[Download article](#) 

Contents 64

1 1. Standard RAG: The Foundational Approach

2 Standard RAG Project Ideas for Practice :

– 1) Legal Document Question Answering System

– 2) Internal Company Knowledge Assistant

5 2. Corrective RAG: The Editor-in-the-Loop

[Show all 64 sections](#)

Retrieval-augmented generation (RAG) is the quiet revolution behind today's best-performing AI systems. ChatGPT's ability to cite sources and enterprise AI's scanning of thousands of documents highlight the need for grounding language models in reality, a role fulfilled by Retrieval-Augmented Generation (RAG). But here's the catch: RAG isn't one-size-fits-all.

Over time, AI researchers have designed specialized RAG architectures, each engineered to overcome different real-world bottlenecks, such as hallucinations, response latency, poor grounding, or limited context. With 25 distinct flavors and variants, how do you know which will take your model from "good" to "production-ready"? In this blog, we're breaking down the 25 types of [RAG architectures](#)  that every AI engineer needs to know.

1. Standard RAG: The Foundational Approach

Let's start with the classic. Standard RAG combines a retriever and a generator. The retriever searches for relevant documents from a knowledge base, and the generator (like GPT-4) crafts a response using this evidence.

Key Features:

- Breaks documents into manageable chunks for retrieval.
- Retrieves only the most relevant information to feed the LLM.
- Ideal for real-time responses (~1–2 seconds).

Use Case: Customer support bots fetching real-time answers from FAQ documents.

Standard RAG Project Ideas for Practice :

1) Legal Document Question Answering System

Standard RAG shines in domains where the primary task is extracting relevant text chunks and generating responses — making it ideal for legal Q&A. In this project, you can [build a chatbot](#)  that answers user questions using retrieval from case law, policies, or contracts. Use chunked legal documents stored in a vector database like FAISS or Weaviate. Standard RAG

keeps the architecture clean: retrieve → generate, no bells and whistles. It's ideal for legally grounded answers where structure and citation are more important than reasoning chains.

2) Internal Company Knowledge Assistant

Standard RAG is perfect for building a fast, no-frills internal helpdesk assistant for small to mid-sized teams. Use it to search across wikis, HR docs, onboarding guides, and tech SOPs. Because the context is mostly factual and direct, basic retrieval + generation covers 80% of the use case. Standard RAG is lightweight, interpretable, and doesn't require additional agent tools or re-ranking. It's perfect for MVPs and quick internal tools where speed and simplicity matter most.

2. Corrective RAG: The Editor-in-the-Loop

Ever felt like the AI "almost got it right"? Corrective RAG is built to fix that. It **uses feedback loops** to refine responses, learning from its own mistakes — or user feedback.

Key Features:

- Iterative corrections over multiple passes.
- Improves user satisfaction by increasing precision.
- Feedback-aware generation loop.

Use Case: E-learning platforms that auto-correct generative quiz answers based on feedback from students or teachers.

Corrective RAG Project Ideas for Practice :

1) Healthcare Chatbot with Medical Document Retrieval

In healthcare, hallucinations can be dangerous — Corrective RAG adds a verification layer to reduce risk. This project builds a medical assistant that retrieves from clinical guidelines, then checks generated answers for factual consistency. Corrective RAG helps flag and correct misleading LLM outputs, keeping responses grounded in real medical sources. Perfect for patient FAQs or provider-support tools where precision and trust are non-negotiable. By rechecking and revising outputs post-generation, the system avoids overconfidence and misinformation.

2) Financial Advisory Assistant for Retail Investors

Financial recommendations must be accurate and well-grounded — Corrective RAG enforces that standard. In this project, users ask investing questions, and the assistant retrieves from SEC filings, financial news, and ETFs docs. The LLM generates an initial answer, and the Corrective RAG pipeline critiques and edits it using retrieved facts. This double-checking mechanism helps mitigate hallucinations common in speculative domains. Great for building trust with non-expert users in regulated industries.

3. Speculative RAG: Draft Fast, Verify Smart

Think of this as a "rough sketch first, polish later" strategy. Speculative RAG uses a small, fast model to draft answers and a larger model to verify and refine them — like a junior-senior editor setup.

Key Features:

- Parallel drafting boosts speed.
- Generalist model ensures final quality.
- Efficient balance between latency and accuracy.

Use Case: News summarization bots that need to be fast but also factual and error-free.

Speculative RAG Project Ideas for Practice :

1) SmartSpec: E-commerce Product Description Generator

A scalable tool that drafts compelling product descriptions using speculative generation, then verifies accuracy against specs and catalogs. Build a product description generator for e-commerce platforms where speed and creativity matter, but accuracy can't be compromised. Speculative RAG allows a smaller, faster model to draft product content while a stronger verifier ensures factual consistency with retrieved specs. This speeds up time-to-market for massive product catalogs while maintaining trust and compliance. It's ideal for balancing scale and quality, especially when rolling out multilingual listings. This setup ensures that the LLM doesn't hallucinate features or violate platform policies.

2) AutoAssist: Verified Email Responder for Customer Support

An AI-powered assistant that drafts lightning-fast support replies and ensures factual precision by verifying internal ticket data. Develop a rapid-turnaround customer email reply tool for support teams dealing with high ticket volume. The speculative model can draft a polite, informative response in real-time, while the verifier model corrects it using relevant ticket history or company policy. This approach enhances agent productivity without compromising brand voice or factual correctness. Speculative RAG is perfect when you want human-like speed but machine-level reliability. It enables fast yet verifiable generation, ideal for semi-automated human-in-the-loop systems.

4. Fusion RAG: Many Sources, One Answer

Why rely on one source when you can consult many? Fusion RAG pulls from multiple retrievers and data sources and fuses the results. This adds context, accuracy, and resilience.

Key Features:

- Diversifies knowledge inputs.
- Dynamically adapts retrieval strategies.
- Less prone to source failures or single-point bias.

Use Case: Financial analysis tools that aggregate insights from regulatory filings, market news, and expert opinions.

Fusion RAG Project Ideas for Practice

1) Cross-Platform Legal Advice Assistant

Create an AI assistant that pulls legal information from multiple platforms, including court rulings, legal databases, and industry-specific news sites. The system should seamlessly integrate these varied data sources and generate accurate legal advice based on the specific query, ensuring that the assistant can handle questions involving complex legal scenarios. The system's strength lies in combining these sources to produce a coherent and precise answer.

2) Multilingual Customer Support Chatbot

Develop a customer support chatbot capable of answering inquiries by retrieving information from multiple languages and cultural contexts. The Fusion RAG approach would allow the chatbot to seamlessly pull information from global support databases, knowledge articles, and localized resources, integrating these different pieces to respond to customers in their preferred language with culturally relevant solutions. This ensures more accurate and contextual responses across diverse customer bases.

5. Agentic RAG: Autonomous Knowledge Seekers

This is where things get exciting. Agentic RAG **uses agents** — independent decision-makers — to dynamically plan, retrieve, and generate based on real-time strategies.

Key Features:

- Modular, agent-based system.
- Parallel task execution.
- Deep understanding of user intent.

Use Case: AI research assistants capable of handling complex, multi-step scientific queries.

Agentic RAG Project Ideas for Practice :

1) Autonomous Policy Research Assistant

Build an agent that helps policy analysts generate reports by autonomously retrieving and comparing data from legislation databases, academic research papers, and current news articles. The agent iteratively reasons about contradictions, aligns sources by credibility, and generates policy briefs with cited sources. The agent plans subtasks — like fetching data from different domains, comparing timelines, and checking source bias — before generating a summary.

2) Competitive Intelligence Agent for Startups

Create an AI agent that continuously monitors competitors' websites, press releases, funding news, job postings, and social media. It synthesizes updates into a weekly market analysis briefing. The agent autonomously sets retrieval targets (e.g., "look for new product launches"), fetches fresh data, summarizes updates, and evaluates shifts in strategy using RAG.

6. Self RAG: The Reflective Thinker

Self-RAG doesn't always go to the knowledge base. Instead, it first uses its own previous outputs as a retrieval base before seeking external help.

Key Features:

- Iterative refinement using internal outputs.
- Improves coherence and consistency.
- Reduces unnecessary retrievals.

Use Case: Long-form storytelling AIs that must maintain narrative consistency across chapters.

Self RAG Project Ideas for Practice

1) Academic Research Critique Assistant

Build an assistant that helps students or researchers review academic papers. The agent retrieves related works, reflects on whether the evidence supports or contradicts the paper's claims, and generates a critique or summary. The agent needs to self-evaluate whether the sources it retrieved are sufficiently relevant or contradictory and refine its output based on that reflection.

2) Ethical Risk Analyzer for AI Policies

Develop a system that evaluates proposed AI ethics policies (e.g., facial recognition rules). The agent retrieves case studies, research, and news examples, then **reflects** on gaps or biases in the evidence it used to make its assessment. Ethical

evaluation demands nuance. The reflective loop allows the agent to reconsider if its retrievals fairly represent both sides of a policy issue and regenerate outputs with balanced sourcing.

7. Adaptive RAG: Smart Retrieval, Every Time

Not all questions need retrieval. Adaptive RAG uses confidence scores to decide when to retrieve and when not to.

Key Features:

- Uses internal model signals to trigger retrieval.
- Balances memory with external knowledge.
- Employs "honesty probes" to avoid hallucinations.

Use Case: Virtual medical assistants that only consult databases for complex cases while answering basic ones using internal memory.

Adaptive RAG Project Ideas for Practice

1) Enterprise Helpdesk Optimizer

Build a smart helpdesk system for internal IT teams that adapts its retrieval approach depending on the user's role and query type. For instance, a DevOps engineer asking about container errors triggers technical log/document retrieval, while a new employee asking about VPN access triggers FAQ and onboarding materials. Adaptive RAG detects the user's context and adapts the retrieval layer accordingly — pulling minimal, user-specific documents for routine queries and deeper technical artifacts for complex issues.

2) Adaptive Clinical Decision Support Tool

Create a tool for doctors that tailors its medical information retrieval based on the severity and ambiguity of a patient's symptoms. For routine diagnoses, it pulls from clinical guidelines; for ambiguous or rare cases, it expands to academic research, clinical trials, and expert interviews. Different cases require different depths of evidence. Adaptive RAG ensures that retrieval is context-sensitive — smart enough to expand or narrow the information scope as needed.

8. REFEED (Retrieval Feedback): Fine-Tuning Without Training

REFEED improves the response **without retraining the model** — just by refining retrieval.

Key Features:

- Reranks answers based on post-retrieval signals.
- Leverages both pre- and post-retrieval content.
- Iteratively improves outputs.

Use Case: Enterprise search engines continuously improve by observing which documents users click on or rate highly.

REFEED RAG Project Ideas for Practice

1) Intelligent Interview Assistant for HR

Build an AI tool that helps HR professionals conduct structured interviews. As the assistant asks or answers questions, it learns from user corrections (e.g., "That's not the right policy" or "This is outdated") and adapts future retrievals accordingly — surfacing more relevant internal documents, policy updates, or candidate evaluation rubrics. The system doesn't need full

retraining; it just updates how it retrieves and re-ranks information based on feedback. Over time, the assistant becomes better at aligning with each HR team's unique style and policy changes.

2) Adaptive Coding Assistant with User Corrections

Create a coding assistant that retrieves from forums, docs, and past projects to suggest code snippets and architectural patterns. When a developer downvotes or rewrites a suggestion, the assistant uses that feedback to adapt its future retrieval behavior — prioritizing newer frameworks, higher-quality examples, or enterprise-specific code. It enables quick personalization without re-training the LLM. Retrieval behavior evolves through feedback, improving over time in a lightweight, non-invasive way.

9. REALM: Retrieval-Aware Language Modeling

REALM is a hybrid beast — **training the retriever during pre-training itself** using masked language modeling.

Key Features:

- Uses Wikipedia-scale corpora during training.
- Trains the retriever using maximum inner product search (MIPS).
- Excellent for open-domain QA.

Use Case: Search-aware assistants that can "prethink" answers using latent document representations, e.g., Google Assistant.

REALM RAG Project Ideas for Practice

1) Long-Form Biography Generator from News Archives

Create an assistant that generates detailed biographies of public figures by retrieving and integrating relevant documents from news archives, interviews, and articles. Training it in a REALM-like manner enables it to learn retrieval patterns that matter for life stories — chronology, event salience, and name disambiguation. This project benefits from pretraining the model to recognize not just what to retrieve, but how the retrieved context shapes long-form narrative structure — something REALM is specifically designed to handle.

2) Domain-Aware Medical Question Answering System

Build a QA system for medical professionals that has been trained using REALM-style techniques — where retrieval from clinical literature is deeply embedded in the model's language modeling pipeline. This allows it to not only retrieve relevant studies but also understand their medical context during response generation. Because REALM integrates retrieval during training, the system develops a nuanced understanding of when and how to retrieve, making it ideal for regulated domains like medicine where evidence must be contextual and precise.

10. RAPTOR: Tree-Based Reasoning

Imagine a mind map of ideas — that's RAPTOR. It clusters content into a hierarchical tree for multi-level retrieval — broad themes on top, specifics below.

Key Features:

- Efficient for answering layered, complex questions.
- Offers tree traversal or collapsed-tree modes.
- Outperforms flat retrieval on nuanced tasks.

Use Case: Legal research bots that retrieve both statutes and case-specific details via abstract-to-detailed branching.

RAPTOR Project Ideas for Practice

1) Complex Financial Risk Assessment Agent

Build an AI agent that helps analysts assess investment risks by breaking down a query into multiple sub-factors e.g., market volatility, regulatory changes, company fundamentals. RAPTOR guides retrieval along each path (e.g., central bank news, industry reports, financial statements), then synthesizes them into an overall risk evaluation. Financial risk is multi-faceted and benefits from reasoning across parallel evidence branches. RAPTOR ensures each component is explored deeply before the final synthesis.

2) AI Debate Coach for Students

Create a tool that helps debate students structure their arguments by tree-exploring a motion (e.g., "AI should be regulated") into subtopics like ethical implications, legal frameworks, and economic impact. The system retrieves arguments and counterarguments for each branch, helping students prepare balanced, high-quality debate points. Debate prep requires reasoning across multiple, often opposing dimensions. RAPTOR's tree-structured retrieval and reasoning help the agent build stronger, multi-perspective arguments.

11. REVEAL: Reason + Vision

REVEAL is a RAG approach tailored for vision-language tasks — think GPT-4V. It blends reasoning, task-aligned thinking, and real-world grounding to reduce hallucinations in visual queries.

Key Features:

- Grounded in real-world visual facts.
- Transparent and interpretable decision-making.
- Few-shot friendly.

Use Case: Visual troubleshooting bots for mechanical systems that "look" at machine parts and suggest repairs using manuals and logs.

REVEAL RAG Project Ideas for Practice

1) Visual Compliance Checker for Manufacturing

Build an AI assistant that audits product designs or packaging images against regulatory and brand compliance checklists. It extracts visual features (like warning labels, layout, logo placement), retrieves documentation about relevant standards (like FDA or ISO), and then flags issues or recommends fixes. This RAG project requires reasoning across visual and textual evidence. The agent must analyze images and align findings with retrieved regulatory guidelines, which is a strength of REVEAL.

2) Educational Tutor for Diagram-Based Learning

Create an intelligent tutor that helps students understand visual concepts in subjects like biology, physics, or geography. When shown a diagram (like the water cycle or a circuit board), it retrieves relevant textbook content, explains the visual step by step, and answers follow-up questions. Learning from diagrams requires the system to interpret visual elements and connect them with explanatory text. REVEAL enables this visual-textual fusion to support rich educational dialogue.

12. REACT: Think, Then Act

REACT (Reason + Act) gives the model a chain-of-thought approach with actionable steps, which is ideal for task-solving. It enables an agent to "think" by reasoning step by step through a query and then "act" by calling tools like search APIs, calculators, databases, or code execution environments. What sets REACT apart is that retrieval is not passive—it becomes an active, decision-driven process where the agent decides what to retrieve, when, and how to use it in context.

Key Features:

- Maintains memory of past steps.
- Thinks logically and acts based on reasoning.
- Enhances transparency and reliability.

Use Case: Coding copilots that debug by generating hypotheses, checking documentation, and modifying code step-by-step.

REACT RAG Project Ideas for Practice

1) Autonomous Data Analyst for Business Teams

Build an AI agent that helps business analysts answer data questions like "Why did Q3 revenue drop in the EMEA region?" It reasons through the query, pulls relevant metrics from dashboards, retrieves meeting notes or CRM entries, runs calculations, and presents a structured response with visual explanations. The agent needs to alternate between reasoning ("I need EMEA revenue trends") and acting ("Query the sales database"), making it a perfect fit for REACT's think-act loop.

2) Legal Assistant for Case Research and Drafting

Create a legal AI that helps lawyers draft arguments or analyze cases. It can retrieve relevant statutes or prior judgments, reason through precedents, highlight contradictions and generate outlines or first drafts of legal briefs. Legal tasks often

require agents to take intelligent actions based on evolving context — searching databases, interpreting clauses, and constructing logical arguments. REACT's iterative decision-making loop supports this complexity.

13. Memo RAG: Build Memory, Beat Latency

Memo RAG is designed for speed and efficiency. It builds a retrieval memory cache over time, remembering which documents were useful in previous queries. Instead of re-querying the entire corpus every time, the system remembers useful past retrievals and reuses high-confidence chunks to save time and improve response consistency. This enables agents to operate with lower latency and better session continuity, especially when users interact repeatedly, or context carries over across turns.

Key Features:

- Memoization of previous retrievals.
- Lower latency and compute costs.
- Great for repeated or similar questions.

Use Case: Customer service bots answering recurring billing or policy-related queries using previously accessed data.

Memo RAG Project Ideas for Practice

1) AI Coach for Continuous Learning

Build a personal learning assistant that helps users master complex subjects like AI, law, or medicine. The assistant remembers previously retrieved explanations, key concepts, and misunderstood topics from earlier sessions, and uses that context to personalize future answers or quiz the learner. Since the learning journey is incremental, the agent benefits from

reusing earlier insights rather than retrieving everything from scratch. This makes interactions faster and more aligned with each user's journey.

2) Executive Briefing Assistant for Busy Leaders

Create a smart briefing tool for executives that summarizes ongoing initiatives, past decisions, and new updates by recalling relevant information across meetings, emails, and reports. When asked "What's the status of Project Titan?", it leverages memory from previous summaries and retrievals to respond instantly. Executives value speed and consistency. Memo RAG enables the system to recall context-rich chunks from prior sessions, ensuring faster responses and reduced repetition.

14. Graph RAG: Connecting the Dots

Most RAG systems retrieve linear chunks of text. **Graph RAG** builds a **knowledge graph** by connecting entities and concepts, allowing the model to reason over structured relationships.

Key Features:

- Structured representation of knowledge.
- Supports reasoning over complex interrelations.
- Enhances explainability.

Use Case: AI legal assistants navigating statutes, case laws, and regulations through a concept graph.

15. Duo RAG: Best of Two Worlds

Duo RAG **combines two generators or retrievers** to improve the output quality. These could be different models or the same model with different prompts or retrieval bases.

Key Features:

- Model diversity reduces hallucinations.
- Enhances robustness.
- Encourages agreement between outputs.

Use Case: Medical chatbots that compare and cross-validate recommendations using two different medical knowledge bases.

16. Context-Aware RAG: Personalized & Persistent

This RAG flavor remembers your context — past conversations, user behavior, and preferences — and tailors its retrieval accordingly.

Key Features:

- Conversational memory.
- User history-aware retrieval.
- Personalized responses.

Use Case: AI tutors adjusting explanations based on a learner's previous questions and mistakes.

17. Ensemble RAG: Let the Experts Decide

Why choose one model when you can use an ensemble? Ensemble RAG routes tasks to multiple RAG pipelines, selecting or combining the best output.

Key Features:

- Combines strengths of different RAG models.
- Voting or ranking-based answer selection.
- Robust fallback mechanism.

Use Case: Enterprise AI systems that need to balance speed, cost, and precision by switching between fast and thorough pipelines.

18. Multimodal RAG: Beyond Text

Multimodal RAG goes beyond textual retrieval and includes images, video, audio, or tabular data as part of its knowledge base.

Key Features:

- Retrieves across modalities.
- Requires multimodal encoders (e.g., CLIP, Flamingo).
- Unlocks cross-domain applications.

Use Case: Virtual museum guides pulling from art images, audio tours, and historical texts to answer visitor queries.

19. Federated RAG: Private & Distributed

When data is decentralized (think hospitals or banks), federated RAG retrieves from local sources without centralizing data.

Key Features:

- Privacy-preserving architecture.
- Supports edge and offline modes.
- Localized context retrieval.

Use Case: Cross-hospital medical diagnosis tools that access records stored on individual servers without violating privacy regulations.

20. Online RAG: Real-Time Learning

Online RAG continuously updates its knowledge base by ingesting real-time documents or events.

Key Features:

- Dynamic ingestion pipelines.
- Near real-time retrievability.
- Prevents information staleness.

Use Case: Stock market analysts that retrieve and summarize the latest filings, tweets, and news alerts in seconds.

21. Modular RAG: Plug-and-Play Architecture

Designed for flexibility, **Modular RAG** allows each component (retriever, reranker, generator, router) to be swapped out independently.

Key Features:

- Swappable modules for different tasks.

- Promotes reusability and experimentation.
- Easier debugging and optimization.

Use Case: AI platforms customizing retrieval pipelines for legal, educational, and healthcare domains by tweaking modules.

22. Multi-Hop RAG: Reason Across Steps

Some questions need multi-step reasoning. Multi-Hop RAG retrieves in multiple rounds — answering intermediate sub-questions before returning a final response.

Key Features:

- Recursive retrieval pipeline.
- Supports decomposition of complex tasks.
- Common in open-domain QA.

Use Case: Academic research assistants answering layered questions like "What led to inflation in 1970s UK and how did policymakers respond?"

23. Tool-Integrated RAG: Retrieval + Action

This version combines RAG with **tool-use abilities**, letting the model perform actions like web searches, calculator functions, or database queries before finalizing the response.

Key Features:

- Retrieval + tool execution loop.

- Dynamic planning and agentic orchestration.
- Hybrid reasoning and computation.

Use Case: AI financial advisors that look up tax laws and compute your refund dynamically.

24. Cascade RAG: Stage-by-Stage Retrieval

Instead of retrieving everything at once, Cascade RAG applies retrieval in stages, refining results at each step with intermediate generators or rerankers.

Key Features:

- Hierarchical retrieval architecture.
- Improves quality for dense knowledge bases.
- Reduces irrelevant hits.

Use Case: Research bots retrieving from patent databases, filtering through inventors, categories, and publication dates step by step.

25. Asynchronous RAG: Parallel + Event-Driven

Finally, Asynchronous RAG allows different components to operate in parallel or trigger on-demand — ideal for distributed or multi-threaded applications.

Key Features:

- Event-driven RAG workflows.

- Parallel retrievers/generators.
- Suits microservice architectures.

Use Case: AI developer assistants that retrieve from documentation, source code, and error logs simultaneously in an IDE.

From the base-level Standard RAG to dynamic agents, graph reasoning, multi-modal learning, and privacy-preserving setups , this compilation of 25 RAG types reflects how rapidly the RAG design space is evolving.

So, where do you go from here?

If you're a developer, data scientist, or AI enthusiast:

- Start with Standard RAG for open-domain QA.
- Experiment with Self RAG or Corrective RAG to improve quality.
- Explore Graph RAG or Multimodal RAG if your domain is complex.
- Combine RAG with agents using REACT or Tool-Integrated RAG to build autonomous systems.

P.S. Want to get hands-on with real RAG projects?

Check out [ProjectPro's end-to-end project repository](#)  for implementations of RAGs, LLM-powered workflows, and agentic AI use cases in ProjectPro's end-to-end project repository — featuring curated datasets, retrieval pipelines, and hands-on notebooks that guide you step by step through building and deploying advanced retrieval-augmented generation systems.

Note: Project ideas for additional RAG architectures will be updated soon; stay tuned and follow for more.



By **Freedium.** – *made with care.*

[About](#)

[Privacy](#)

[Terms](#)

[RSS](#)

[GitHub](#)

[Codeberg](#)



· *Freedium*^{beta}

